

Linux—*user-mode-linux.sourceforge.net*). Each emulated network device is a virtual GNU/Linux system. Since GNU/Linux is shipped with software supporting most of the network protocols, any machine can be configured to act as a bridge/switch or as a router. Each virtual machine has a console (terminal window) with a root shell.

Installing and setting up Netkit

The hardware requirements for Netkit are pretty basic:

- ≥ 600 MHz CPU
- ~10 MB memory for each VM (depending on the VM configuration)
- ~600 MB of disk space + ~1-20 MB for each VM (depending on the usage of the VM)

As for software requirements, you'll need a GNU/Linux installation and of course, you can download Netkit from www.netkit.org if it's not available in your distro's online software repositories. [Netkit works fine on many distributions—see wiki.netkit.org/index.php/Status].

Download the following three files: *netkit-XY.tar.bz2*, *netkit-filesystem-FX.Y.tar.bz2* and *netkit-kernel-KX.Y.tar.bz2*.

Unpack them in the same location:

```
slyrux@gruboc~$ mkdir ~/netkit
// Follow the following order in extraction of archives
slyrux@gruboc~$ tar -xjf netkit-XY.tar.bz2 -C ~/netkit
slyrux@gruboc~$ tar -xjf netkit-filesystem-FX.Y.tar.bz2 -C ~/netkit
slyrux@gruboc~$ tar -xjf netkit-kernel-KX.Y.tar.bz2 -C ~/netkit
```

Now it requires you to set up some environment variables to make Netkit work. Append the following lines to your *~/.bashrc*:

```
export NETKIT_HOME=~/.netkit
export PATH=$PATH:$NETKIT_HOME/bin
export MANPATH=$NETKIT_HOME/man
```

You can now check your Netkit installation by opening a new terminal running the following *check_configuration.sh* bash script.

```
slyrux@gruboc~$ cd $NETKIT_HOME
slyrux@gruboc~$ ./check_configuration.sh
```

If it succeeds, your Netkit installation is successful.

An introduction to Netkit commands

We will now look into some of the Netkit commands used to start, configure and monitor Netkit virtual machines and the virtual labs.

- *vstart* starts a new virtual machine
- *vlist* lists the currently running virtual machines
- *vconfig* attaches network interfaces to running VMs
- *vhalt* gracefully halts a virtual machine
- *vcrash* causes a virtual machine to crash
- *vclean* is the "panic command" to clean up all Netkit

processes (including VMs) and configuration settings on the host machine

The Netkit lab is an interesting facility that comes inbuilt with the Netkit toolkit. While emulating a large network, it is harder to configure each of the VMs individually, every time you want to conduct a new experiment. Netkit lab facilities help in setting up complex labs consisting of several virtual machines. Here are some commands to manipulate and keep track of the running labs:

- *lstart* starts a Netkit lab
- *lhalt* gracefully halts all the VMs of a lab
- *lcrash* causes all the VMs of a lab to crash
- *lclean* removes temporary files from a lab directory
- *linfo* provides information about a lab without starting it
- *ltest* allows you to run tests to check that the lab is working properly

Starting virtual hosts

Each of the VMs represents a network device or a host. In order to start a virtual machine with the name 'pc1', use the following command:

```
slyrux@gruboc~$ vstart pc1
===== Starting virtual machine 'pc1' =====
Kernel: /home/slyrux/netkit/kernel/netkit-kernel
Modules: /home/slyrux/netkit/kernel/modules
Memory: 16 MB
Model fs: /home/slyrux/netkit/fs/netkit-fs
Filesystem: /home/slyrux/pc1.disk
Hosts at: /home/slyrux
```

```
Running ==> /home/slyrux/netkit/kernel/netkit-kernel modules=/home/
slyrux/netkit/kernel/modules name=pc1 title=pc1 umid=pc1 mem=20M
ubd0=/home/slyrux/pc1.disk /home/slyrux/netkit/fs/netkit-fs root=98:1
uml_dir=/home/slyrux/netkit/mconsole hosthome=/home/slyrux quiet
con0=xterm con1=null SELINUX_INIT=0
```

A console terminal corresponding to pc1 will pop up. You can create any number of virtual hosts using the *vstart* command. We will use options like—*eth0*—*eth1* to specify the network interface and its connection. For example:

```
slyrux@gruboc~$ vstart pc1 --eth0=A
```

```
slyrux@gruboc~$ vstart pc2 --eth0=A
```

Here we have two machines pc1 and pc2. The—*eth0* argument corresponds to the network interface, while 'A' in the—*eth0=A* argument specifies the collision domain of the network—i.e., in the above example, pc1 and pc2 are connected on the same network family as if they are interconnected through a network hub.

If there is a pc3—*vstart pc3--eth0=B*—it means pc3 is connected to a different network hub and pc3 is not reachable from pc1 or pc2. In order to bridge both hubs, we require a router in between.